Java Backend Architecture

Interview Series

Chapter 11.1 Optimistic Concurrency Control (OCC)

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

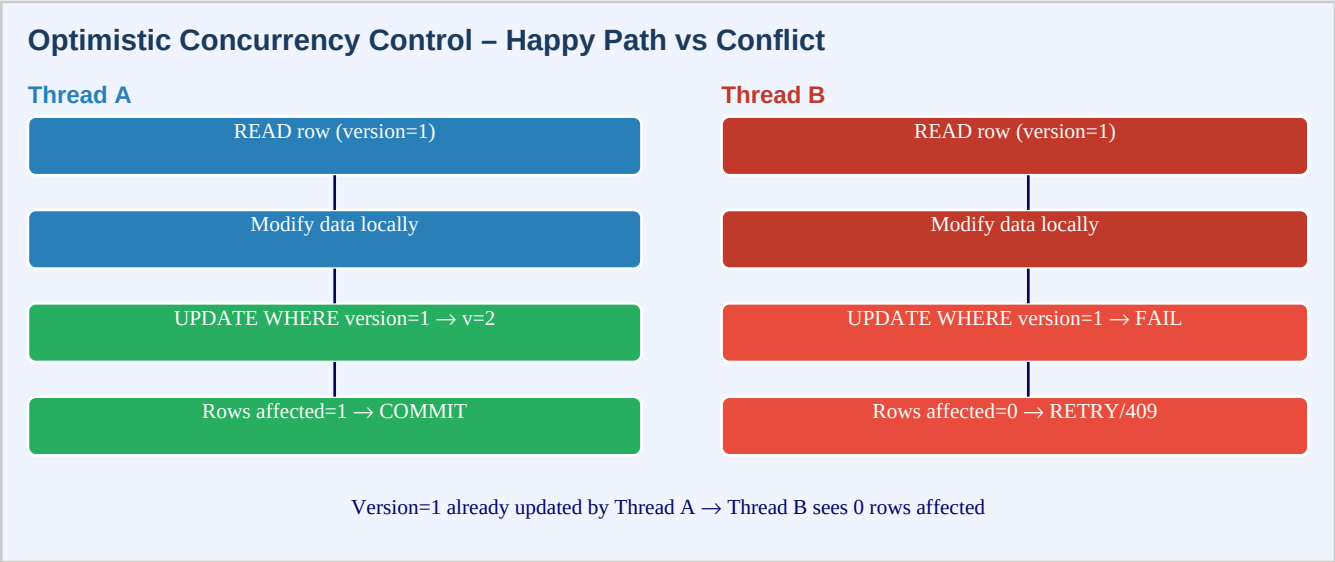
Chapter 11.1 – Optimistic Concurrency Control (OCC)

Introduction

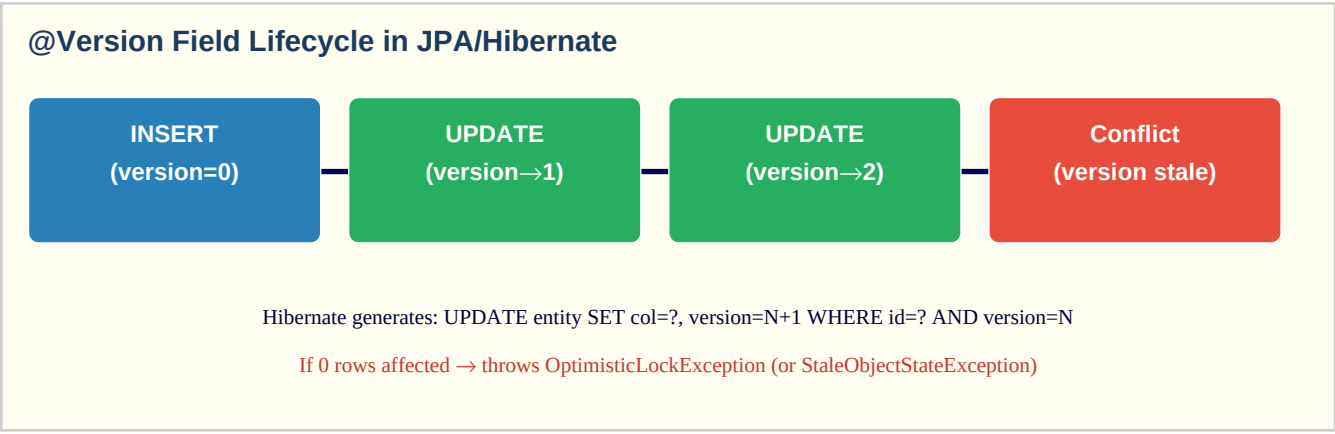
Optimistic Concurrency Control (OCC) assumes that conflicts between concurrent transactions are rare and allows multiple readers and writers to proceed without blocking each other. Instead of locking a row when it is read, OCC records a version number or timestamp with the row. When a writer attempts to commit, it checks that the version still matches what was read — if another transaction has modified the row in the meantime, the version will have changed and the write is rejected with a conflict error.

OCC is the right choice for read-heavy, low-contention workloads: e-commerce product catalogs, user profile updates, document editing. It avoids the overhead of database row locks held for the duration of a business transaction. The tradeoff is that conflicts must be caught and handled at the application layer — typically by retrying with fresh data.

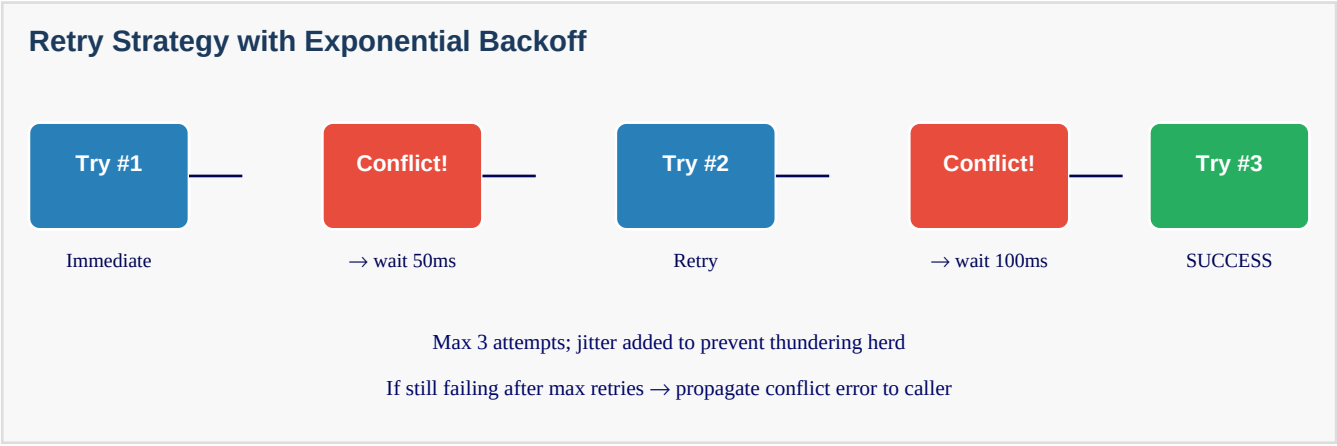
OCC Happy Path vs Conflict



@Version Field Lifecycle



Retry Strategy



Key Definitions

OCC	Optimistic Concurrency Control — allow concurrent access, detect conflicts at commit
@Version	JPA annotation marking the version field; Hibernate manages increment automatically
OptimisticLockException	JPA exception thrown when version mismatch detected (0 rows updated)
StaleObjectStateException	Hibernate-specific exception for stale version on merge/update
version column	Integer or timestamp column appended to UPDATE WHERE clause as a guard
conditional UPDATE	UPDATE … SET … WHERE id=? AND version=? — atomic OCC check at DB level
rows affected	JDBC update count; 0 means WHERE clause matched nothing (version mismatch)
retry	Re-read fresh data and re-attempt the business logic after a conflict
backoff	Waiting before retrying; exponential backoff reduces contention under load
LWW (Last-Write-Wins)	No conflict detection — last writer overwrites silently; dangerous for OCC-worthy data
read-modify-write	Classic pattern OCC protects: read data, modify locally, write back with version check
MVCC	DB-level multi-version concurrency control; OCC is application-level analogue
fencing token	Monotonically increasing token issued with lock; storage rejects old tokens
idempotency key	Client-supplied key to detect and deduplicate duplicate/retried requests

OCC vs Pessimistic Locking

Aspect	Optimistic (OCC)	Pessimistic (SELECT FOR UPDATE)

Lock held	None	From SELECT to COMMIT/ROLLBACK
Best for	Low contention, read-heavy	High contention, short txns
Conflict handling	Detect at commit, retry	Wait or timeout at SELECT
Throughput	High (no waiting)	Lower (serialized writers)
Starvation risk	Possible under high contention	Possible with long lock holders
DB overhead	Minimal (version column only)	Lock manager overhead
Spring/JPA	@Version on entity	LockModeType.PESSIMISTIC_WRITE

Code Examples

1. JPA entity with @Version

```
@Entity @Table(name = "products") public class Product { @Id @GeneratedValue(strategy = GenerationType.UUID) private String id; private String name; private BigDecimal price; private int stockQuantity; @Version private Long version; // Hibernate manages this; do NOT update manually // getters/setters } // DDL equivalent: // CREATE TABLE products ( // id VARCHAR(36) PRIMARY KEY, // name VARCHAR(255), // price NUMERIC(19,2), // stock_quantity INT, // version BIGINT NOT NULL DEFAULT 0 // ); // Hibernate generates on update: // UPDATE products SET name=?, price=?, stock_quantity=?, version=2 // WHERE id=? AND version=1 // If 0 rows affected → OptimisticLockException
```

2. Service layer with retry on conflict

```
@Service @Transactional public class InventoryService { private final ProductRepository repo; private static final int MAX_RETRIES = 3; private static final long BASE_DELAY_MS = 50; public Product decrementStock(String productId, int qty) { int attempt = 0; while (true) { try { return doDecrement(productId, qty); } catch (OptimisticLockException | ObjectOptimisticLockingFailureException ex) { attempt++; if (attempt >= MAX_RETRIES) { throw new ConcurrencyConflictException( "Could not update product after " + MAX_RETRIES + " attempts", ex); } long delay = BASE_DELAY_MS * (1L << attempt) // exponential backoff + ThreadLocalRandom.current().nextLong(20); // jitter try { Thread.sleep(delay); } catch (InterruptedException ie) { Thread.currentThread().interrupt(); throw new RuntimeException(ie); } // Important: clear the EntityManager so stale state is evicted entityManager.clear(); } } } @Transactional(propagation = Propagation.REQUIRES_NEW) private Product doDecrement(String productId, int qty) { Product p = repo.findById(productId) .orElseThrow(() -> new EntityNotFoundException(productId)); if (p.getStockQuantity() < qty) { throw new InsufficientStockException(); } p.setStockQuantity(p.getStockQuantity() - qty); return repo.save(p); // triggers version check on flush } }
```

3. Pure JDBC conditional UPDATE (no ORM)

```
// Without Hibernate — roll your own OCC at JDBC level @Repository public class ProductJdbcRepository { private final JdbcTemplate jdbc; public boolean updatePriceWithVersion(String id, BigDecimal newPrice, long currentVersion) { int rows = jdbc.update( "UPDATE products SET price = ?, version = version + 1 " + "WHERE id = ? AND version = ?", newPrice, id, currentVersion ); return rows == 1; // false = someone else updated first } } // Usage: Product p = findById(id); boolean success = repo.updatePriceWithVersion(p.getId(), newPrice, p.getVersion()); if (!success) { throw new OptimisticLockConflictException("Product was modified concurrently"); }
```

4. Spring Data JPA — locking via repository

```
public interface ProductRepository extends JpaRepository<Product, String> { // Spring Data handles @Version automatically on save() // For explicit lock on fetch: @Lock(LockModeType.OPTIMISTIC_FORCE_INCREMENT) @Query("SELECT p FROM Product p WHERE p.id = :id") Optional<Product> findByIdForUpdate(@Param("id") String id); // OPTIMISTIC → reads current version, checks at commit (default @Version behaviour) // OPTIMISTIC_FORCE_INCREMENT → always increments version even if entity unchanged // (useful for parent entity when only child changed) }
```

5. REST API — return version to client, accept it on PUT

```
// GET /api/products/{id} → response includes version { "id": "prod-123", "name": "Running Shoes", "price": 89.99, "version": 5 } // Client stores version, sends it back on update: // PUT /api/products/{id} { "name": "Running Shoes Pro", "price": 99.99, "version": 5 // must match current DB version } @PutMapping("/api/products/{id}") public ResponseEntity<Product> update( @PathVariable String id, @RequestBody ProductUpdateRequest req) { try { Product updated = service.update(id, req); return ResponseEntity.ok(updated); } catch (OptimisticLockException ex) { return ResponseEntity.status(HttpStatus.CONFLICT) .body(service.findById(id)); // return current state for merge UI } }
```

6. @Retryable with Spring Retry

```
// pom.xml: spring-retry + spring-aspects @Service public class OrderService { @Retryable( retryFor = { OptimisticLockException.class, ObjectOptimisticLockingFailureException.class }, maxAttempts = 3, backoff = @Backoff(delay = 50, multiplier = 2, random = true) ) @Transactional public Order placeOrder(CreateOrderCommand cmd) { // read, validate, write — if OLE thrown, Spring retries Order order = buildOrder(cmd); return orderRepo.save(order); } @Recover public Order recoverFromConflict(OptimisticLockException ex, CreateOrderCommand cmd) { // Called if all retries exhausted throw new ConcurrencyConflictException("Order placement failed after retries", ex); } }
```

Interview Questions & Answers

Q1. What is Optimistic Concurrency Control and when should you use it?

OCC allows multiple transactions to proceed concurrently without locking rows. Each transaction reads a version number with the data and checks at write time that the version hasn't changed. If it has, the write is rejected and retried. Use OCC when: (1) conflicts are rare (low-contention), (2) transactions are long (holding DB locks for seconds is too expensive), (3) you prioritize throughput over strict serialization. Classic use cases: product catalog updates, user profile edits, document saves where concurrent edits are uncommon.

Q2. How does Hibernate implement @Version?

Hibernate adds the version column to every UPDATE statement as a WHERE predicate: `UPDATE entity SET col=?, version=N+1 WHERE id=? AND version=N`. If the UPDATE returns 0 rows affected (another transaction already incremented the version), Hibernate throws OptimisticLockException (or StaleObjectStateException on merge). The version is incremented automatically by Hibernate — never modify the @Version field manually. Supported types: Integer, Long, Short, Timestamp, and Instant.

Q3. What exception is thrown on a version conflict in JPA and how do you handle it?

JPA throws javax.persistence.OptimisticLockException. Hibernate throws a subclass StaleObjectStateException, which Spring wraps as ObjectOptimisticLockingFailureException. Handling: (1)

catch the exception, (2) clear the EntityManager to evict stale state, (3) re-read fresh data, (4) re-apply business logic, (5) retry the transaction. After N retries (typically 3), propagate a ConcurrencyConflictException to the caller (which maps to HTTP 409 Conflict at the API layer).

Q4. What is the difference between @Version and SELECT FOR UPDATE?

@Version (OCC): no DB lock held; conflict detected only at commit time; other threads can read and modify the row freely; high throughput. SELECT FOR UPDATE (pessimistic): acquires a row-level exclusive lock at SELECT time; other writers block until the lock is released; no conflict at commit (guaranteed exclusive access); lower throughput under contention. Rule of thumb: OCC for read-heavy, low-conflict workloads; SELECT FOR UPDATE for guaranteed-exclusive, write-heavy, or short-duration transactions.

Q5. What is OPTIMISTIC_FORCE_INCREMENT and when is it useful?

OPTIMISTIC_FORCE_INCREMENT increments the version of an entity even if the entity itself was not modified. This is useful for aggregate root patterns in DDD: if a parent entity owns a collection of children, modifying a child doesn't change the parent's column values but you want to bump the parent's version to signal that the aggregate changed. Example: Order has OrderItems — updating an OrderItem should also bump the Order version to prevent concurrent reads of the Order from missing the item change.

Q6. How do you implement OCC without Hibernate using plain JDBC?

Add a `version` BIGINT column to the table with DEFAULT 0. On UPDATE: `UPDATE table SET ..., version = version + 1 WHERE id = ? AND version = ?`. Check the JDBC update count: if 0, a conflict occurred. Return a boolean or throw a ConcurrencyConflictException. On INSERT: include version=0. Include the version in all API responses so clients can send it back. This is identical to what Hibernate does under the hood — you just manage it manually.

Q7. How should you expose version to API clients?

Include `version` in the GET response body (or as an ETag header). The client stores the version and sends it back on PUT/PATCH. The server checks that the version matches the current DB value before applying the update. On conflict, return HTTP 409 with the current resource state so the client can show a 'someone else changed this' UI. Alternatively, use the ETag header + If-Match mechanism (HTTP standard for the same concept).

Q8. What is a 'lost update' anomaly and how does OCC prevent it?

A lost update occurs when two transactions read the same row, both modify it independently, and the second write silently overwrites the first. Example: both A and B read balance=100, A adds 50 (writes 150), B adds 30 (writes 130 — A's change is lost). OCC prevents this because A and B both read version=1. A increments to version=2 successfully. B's UPDATE WHERE version=1 fails (version is now 2) → B retries with fresh data (balance=150) and writes 180. No update is lost.

Q9. When does OCC perform worse than pessimistic locking?

OCC degrades under high contention: many writers competing for the same row means frequent conflicts, retries, and wasted work. Each failed attempt reads + computes + tries to write — all CPU and DB work is thrown away. If the conflict rate is high (>10-20%), the cumulative retry overhead may exceed the cost of pessimistic locking. Example: a viral flash sale where thousands of threads decrement the same inventory counter simultaneously — pessimistic locking or a Redis counter is more appropriate.

Q10. How does the retry loop work and what are the risks?

Re-read fresh data, re-apply business logic, re-attempt the write. Risks: (1) Infinite retry loop — always cap at max attempts (e.g., 3). (2) Thundering herd — all retries fire simultaneously after the same backoff; add random jitter to spread them out. (3) EntityManager stale state — must call entityManager.clear() or use a new transaction scope so Hibernate re-fetches from DB, not L1 cache. (4) Non-idempotent side effects — if

the business logic sends an email, retrying it sends duplicate emails; separate side effects from the OCC-protected write.

Q11. What is the EntityManager first-level cache and why must you clear it on retry?

The EntityManager (L1 cache) caches entity state for the current transaction/session. If you catch an OptimisticLockException and retry within the same EM, it will re-use the stale entity from cache without hitting the DB. You must either: (1) call entityManager.clear() to evict all entities, (2) call entityManager.refresh(entity) for the specific entity, or (3) start a new transaction (REQUIRES_NEW propagation) so a fresh EM is created. Failing to do this causes infinite retries of the same stale state.

Q12. How does Spring Retry's @Retryable simplify OCC retry logic?

@Retryable intercepts exceptions via AOP and transparently retries the annotated method. Configure `retryFor` with OptimisticLockException/ObjectOptimisticLockingFailureException, `maxAttempts`, and `@Backoff` for delay/multiplier. The `@Recover` method is called if all retries are exhausted. Important: the method must be called from a different Spring bean (proxy-based AOP), so self-calls within the same class won't be intercepted. Each retry starts a fresh @Transactional context (new EM) if the method is annotated with both.

Q13. Can @Version use a timestamp instead of a counter?

Yes — @Version supports java.util.Date, java.sql.Timestamp, and java.time.Instant. Timestamp versioning uses the current time as the version: the UPDATE WHERE clause checks timestamp equality. Risk: if two writes happen within the same timestamp granularity (milliseconds on some DBs), the check may pass incorrectly — a 'phantom' lost update. Integer/Long counters are strictly safer since they increment deterministically. Use timestamp versions only if you also need a 'last modified' audit trail and the timestamp precision is fine (>1ms granularity).

Q14. How do you test OCC behavior in a Spring Boot integration test?

Use two concurrent transactions: (1) Fetch entity in TX1, fetch same entity in TX2. (2) Commit TX1 (increments version). (3) Commit TX2 — should throw OptimisticLockException. In Spring: use TransactionTemplate or @Transactional with REQUIRES_NEW to create independent transactions. Use ExecutorService with two threads. Assert that the second thread throws ObjectOptimisticLockingFailureException. For retries: verify via mock that the repository is called N times before the final success/failure.

Q15. What is the difference between OCC at the application level vs DB-level MVCC?

Database MVCC (e.g., PostgreSQL) maintains multiple row versions internally so readers never block writers and vice versa. Each transaction sees a consistent snapshot of the DB at its start time. This is transparent to the application. Application-level OCC adds an explicit version column and checks it in the SQL WHERE clause — it's a manual implementation of a similar concept. The two work at different layers: MVCC is for read isolation, OCC is for write conflict detection. Both can coexist in the same system.

Q16. How do you handle OCC in a microservices architecture where data spans services?

Each service manages its own version column within its bounded context. For cross-service operations (e.g., reserve inventory AND create order), use the Saga pattern instead of distributed transactions: each service performs its own OCC-protected local write and publishes an event. If a downstream service fails, compensating transactions roll back upstream changes. Never do distributed locking across microservices — it creates tight coupling. OCC is per-aggregate, Saga coordinates across aggregates.

Q17. What happens if the @Version column is NULL in the database?

If the @Version field is null, Hibernate skips the version check — the UPDATE WHERE clause omits the version predicate, behaving like a regular (non-OCC) update. This can happen if rows were inserted without

the version column or from legacy data. Prevention: add NOT NULL DEFAULT 0 to the DDL. Hibernate treats null as 'unversioned' rather than throwing. In production, ensure all rows have a non-null version — a data migration may be needed for legacy data.

Q18. Describe a scenario where OCC causes a livelock.

Livelock: two threads continuously conflict and retry, each blocking the other's retries. Example: Thread A reads version=1, Thread B reads version=1. A writes (v→2), B fails and retries. B reads version=2, A reads version=2. A writes (v→3), B fails again. This cycle can continue. Solutions: (1) randomized exponential jitter breaks symmetry. (2) Cap retries + fail loudly after N attempts. (3) If livelock is frequent, switch to pessimistic locking or a queue-based serialization for that resource.

Q19. How does OCC interact with database connection pooling?

OCC retries should use a fresh transaction (Propagation.REQUIRES_NEW), which acquires a fresh connection from the pool. If the pool is exhausted during high-conflict periods (many concurrent retries), threads will block waiting for connections — a connection pool starvation scenario. Mitigations: (1) keep retry logic outside of pooled connection context (retry loop is outside @Transactional, each attempt starts a new one). (2) Size the pool for peak retry concurrency. (3) Use async retries to release the thread rather than blocking.

Q20. What is a conditional write in DynamoDB and how does it relate to OCC?

DynamoDB's `ConditionExpression` is the equivalent of OCC at the document level. Example: `attribute_exists(version) AND version = :expected_version`. If the condition fails, DynamoDB throws ConditionalCheckFailedException — equivalent to 0 rows affected in JDBC. The application catches this and retries. DynamoDB also supports `TransactWriteItems` for multi-item atomic OCC. This is pure OCC with no DB-level locks, consistent with DynamoDB's distributed, lock-free architecture.

Q21. How would you implement OCC for a collaborative document editor?

Each document has a version counter. When a user opens the document, they receive the current version. When saving, they submit their changes + the version they started from. The server checks: if the DB version equals the submitted version, apply the change and increment version. If not (someone else saved while you were editing), return 409 with the current document content. The client shows a merge UI: 'Your changes' vs 'Current version'. For real-time collaboration with multiple simultaneous editors, OCC alone is insufficient — CRDTs or Operational Transformation (OT) are needed.